

Fewer Tokens, Better Action: GPT-6 Astra Robot Agents with 14% Higher Success Rate but 65% Fewer Tokens

Ruiyang Si¹ Jianxin Bi^{1,2} Shunyu Yang¹ Rui Ni¹
Wenbo Huang¹ Qiang Wang¹ Shulong Jiang¹ Duomin Wang³
Xiuyu Li⁴ Haiwen Feng⁴ Zhen Dong³ Daquan Zhou¹

¹Peking University, ²National University of Singapore, ³NVIDIA, ⁴Impossible Research

Abstract

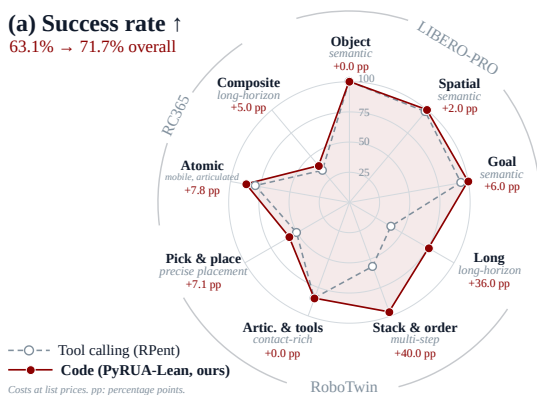
Vision language model (VLM) agents can control robots through visual feedback and action primitives, but repeated model invocations and redundant observations incur substantial token overhead. We introduce PyRUA-Lean, an interactive code-execution framework that couples feedback-driven primitive composition with selective observation: the agent composes classical robot primitives and learned vision-language-action (VLA) policies into Python cells that perform conditional checks and local retries, returning only explicitly requested images and state feedback for replanning. Across 700 simulated task instances from LIBERO-PRO, RoboTwin 2.0, and RoboCasa365, we compare PyRUA-Lean with a tool-calling baseline using the same GPT-6 Astra planner and underlying robot primitives. Under equal LLM-call budgets, PyRUA-Lean increases overall success from 63.1% to 71.7%. On instances solved by both agents, it uses 49% fewer LLM calls and 65% fewer input tokens.

 **Project Page:** <https://dagroup-pku.github.io/PyRUA-Lean/>

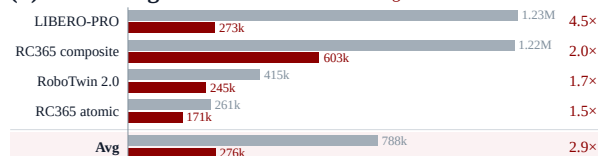
 **GitHub Repo:** <https://github.com/DAGroup-PKU/PyRUA-Lean>

 **Corresponding Author:** Daquan Zhou

(a) Success rate ↑
63.1% → 71.7% overall



(b) Token usage ↓ 65% fewer tokens on average



(c) Cost ↓ 2.2× cheaper on average

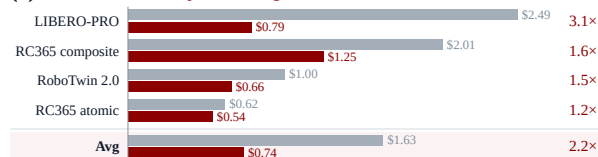


Figure 1 PyRUA-Lean matches or exceeds tool calling’s success rate on each sub-suite and costs less on every benchmark. (a) Success rate on all 700 instances, tagged by task property (semantic: perturbed layouts, objects, goals); RoboTwin 2.0 split by skill from task names. (b, c) Input tokens and GPT-6 Astra list-price dollars per solved episode, on instances both agents solved; Avg: mean over all of them. RC365: RoboCasa365.

1 Introduction

Vision-language models (VLMs) can perform robot manipulation by interpreting visual observations and robot state, then invoking motion, perception, and grasping primitives, including vision-language-action (VLA) policies [1, 10, 26, 33]. Their efficiency depends partly on how these primitives are exposed to the planner. In a tool-calling interface, an operation that depends on a previous tool’s result generally requires another VLM invocation. Intermediate results also accumulate in the conversation and are processed in subsequent calls; in systems such as RPent [33], motion primitives automatically return multiple camera images at the end of each step of execution. Tasks requiring repeated localization, motion, and recovery can therefore incur substantial inference overhead, even when individual primitives are reliable.

Executable code provides a way to reduce this overhead. A program can compose primitives, inspect their outputs, and execute conditional branches or retries before returning control to the VLM. Code-based robot control is well established: Code as Policies and ProgPrompt generate robot programs from language instructions [14, 25], while more recent systems incorporate execution feedback or refine programs across trials [9, 16]. Outside robotics, code-based agent interfaces have also shown benefits in task completion and context efficiency [2, 27, 29]. These findings motivate our central question: *how can a VLM agent compose robot primitives to reliably complete tasks while reducing token consumption through selective observation and context management?*

We introduce **PyRUA-Lean** (Python for Lean Robot-Use Agents), an interactive code-execution interface for feedback-driven primitive composition and selective observation. The agent generates Python programs that compose robot primitives, check intermediate outcomes, and condition subsequent actions on execution feedback. Intermediate results remain available in a persistent program state, while only explicitly printed outputs and requested camera images are returned to the VLM. This allows the agent to handle intermediate execution steps without repeated model invocations and to control which observations enter its context.

PyRUA-Lean builds on RPent’s robot stacks and primitive implementations [33]. We compare it with RPent’s tool-calling agent under the same GPT-6 Astra planner, frozen VLA policies, evaluation task instances, and LLM-call budgets. Both agents operate without cross-episode memory. The comparison evaluates the interfaces as a whole, including their support for control flow, persistent program state, and observation delivery. Experiments cover 700 simulated task instances from LIBERO-PRO, RoboTwin 2.0, and RoboCasa365.

Our contributions can be summarized as:

- **An interactive interface for robot primitive composition.** We introduce PyRUA-Lean, a Python execution interface built on RPent’s robot stack. It supports feedback-driven primitive composition, persistent program state, and selective observation delivery.
- **Comprehensive evaluation on three robot benchmarks.** Across 700 simulated instances, PyRUA-Lean improves overall success by approximately 14% relative to tool calling. On jointly solved instances, it uses 49% fewer VLM calls and 65% fewer input tokens.
- **Analysis of efficiency gains and limitations.** Traces and ablations identify fewer LLM calls as the main source of token savings, which persist without VLA policies or operating guides but diminish on VLA-dominated tasks, and illustrate geometric computation and scripted recovery.

2 Related Work

VLM-based robot control. Language models have been used to select robot skills, interpret execution feedback, and generate motion commands. SayCan grounds skill selection in language instructions and learned affordances [1], while Inner Monologue incorporates environment feedback into planning [10]. More recent systems support finer-grained control: Show-Harness exposes discrete semantic actions to a VLM [8], and GPT-6 Astra has been evaluated as an embodied policy that generates or corrects robot actions [26]. VLA models provide learned visuomotor capabilities for language-conditioned control [3, 4, 13, 23]. Harness VLA integrates frozen VLA policies with analytic primitives through RPent, a memory-guided agent framework [33]. PyRUA-Lean builds on RPent’s robot stacks and primitive implementations, exposing these capabilities

through an interactive Python interface. We compare this interface with RPent’s tool-calling interface using the same planner and underlying primitives, with cross-episode memory disabled in both agents.

Code-based robot control. Code as Policies and ProgPrompt generate executable robot programs from language instructions [14, 25], while VoxPoser generates code that constructs spatial value maps for motion planning [11]. Subsequent work explores execution feedback and program adaptation. CaP-X benchmarks coding agents across primitive abstraction levels and single- or multi-turn interaction modes [9], and VLCP periodically regenerates a control function from updated observations within an episode [17]. Other systems accumulate reusable capabilities across trials: Voyager develops a code-based skill library in Minecraft [28], ASPIRE refines robot programs and consolidates experience into reusable skills [16], and RoboRSI converts established tool-use workflows into executable routines as part of a robot self-improvement system [19]. These studies establish program synthesis, feedback-driven execution, and skill reuse as complementary approaches to embodied control. Our focus is the success and inference cost of interactive code execution relative to tool calling over the same robot primitives, including VLA policies. We evaluate this comparison without cross-episode skill accumulation, while allowing state and code reuse within each episode.

Code-based interfaces for efficient agents. Research on general-purpose agents provides a broader motivation for this comparison. ReAct interleaves reasoning with environment interaction [32], and Toolformer studies the use of external APIs by language models [24]. CodeAct shows that executable Python can improve agent performance relative to text or JSON action interfaces [29], while SWE-agent demonstrates the importance of interface design for software-engineering agents [31]. More recent systems use code to compose tool calls and process intermediate results outside the model context, reducing the information returned to the VLM [2, 27]. PyRUA-Lean applies these interface principles to robot manipulation, where observations are visual and primitive execution may fail. We measure task success together with LLM-call usage, input-token consumption, and inference cost, and analyze how programmatic composition and observation passing contribute to the gains.

3 PyRUA-Lean: Primitive Composition with Selective Observation

PyRUA-Lean is an interactive code-execution interface for VLM-based robot agents. It supports two complementary mechanisms: composing robot primitives into programs that respond to execution feedback, and controlling which intermediate results and visual observations enter the VLM context. The interface builds on RPent’s robot stacks and primitive implementations [33], including classical motion and perception primitives and learned VLA policies.

3.1 Robot Interface and Execution Model

The robot is exposed as a Python object, `robo`, whose methods invoke primitives and return structured results, such as object positions and motion outcomes. The agent receives an API reference (Table 1 lists the primitives) and interacts with the robot through a `python(code)` tool. At each interaction, the VLM generates a code cell, the runtime executes it, and the returned feedback informs the next code cell. A persistent Python namespace retains variables and helper functions across cells within an episode. Our evaluation uses no cross-episode memory.

An uncaught exception terminates the current cell and returns its traceback to the VLM. Variables assigned before the exception remain in the persistent namespace, allowing the agent to revise the failed operation in a subsequent cell. Before each primitive that acts on the robot, the runtime terminates the cell if the task has already succeeded. An external watchdog enforces the two-hour episode limit by interrupting any active primitive and stopping the planner. We impose no separate per-cell execution limit; in our setup, the Codex CLI returns a tool-call timeout after 300 s without cancelling the cell, and subsequent cells are queued.

3.2 Feedback-Driven Primitive Composition

Each cell can compose multiple primitives and condition subsequent operations on their outcomes. For example, a program may approach an object, attempt a grasp only if the approach succeeds, and retry or stop based on the resulting state. These checks and branches execute within the cell without additional

PyRUA-Lean: Primitive Composition with Selective Feedback

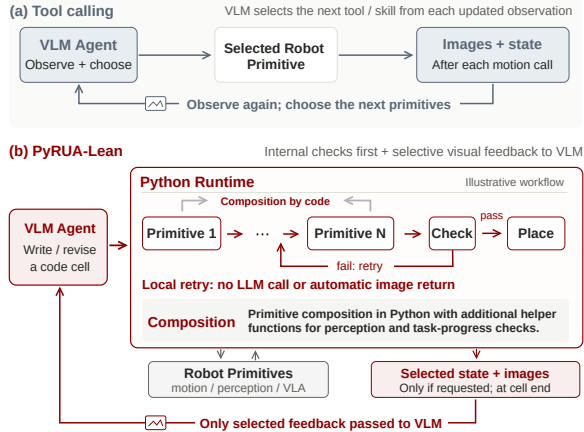


Figure 2 PyRUA-Lean combines primitive composition with selective feedback. (a) The tool-calling agent invokes robot primitives through tool calls. Operations that depend on preceding results generally require another VLM turn, and motion calls automatically return images and state. (b) PyRUA-Lean instead generates Python cells that compose robot primitives with helper functions, conditional checks, and local retries. Intermediate execution stays within the runtime, while explicitly requested images and state messages are recorded and returned at cell end for replanning. (c) A recorded placement example illustrates both mechanisms: one PyRUA-Lean cell replaces baseline steps 14–17, reducing four LLM turns to one. The cell computes a placement target from scene geometry (compose), conditionally executes lowering and release (compose), and requests an image only if the task remains unfinished (select). This successful execution returns five printed lines and no image, demonstrating how programmatic composition and selective feedback reduce repeated model interaction. Appendix A follows the whole episode.

LLM invocations. At the end of the cell, the returned feedback allows the LLM to revise subsequent actions, combining program-level feedback with LLM-level replanning.

Beyond composing primitive calls, programs can *compose* task-specific computations and control routines from existing observations and primitives, as illustrated by the geometric computation in Figure 2(c). For example, the agent computes the bowl’s base height from a world-coordinate map to determine a placement target that no individual primitive directly returns. It can also define helper functions for repeated operations, such as a guarded descent. Computed values and helper functions remain available to later cells through the persistent namespace.

3.3 Selective Observation and Context Management

Primitive results remain available in the runtime without being individually added to the LLM conversation. The agent specifies which feedback to return through explicit output statements and camera requests such as `robo.show`. Requested images and state messages are recorded at the points specified in the code and returned together at the end of the cell. In the episode summarized in Appendix A, cell 5 requests a camera image after transporting the bowl. The placement cell shown in Figure 2(c) requests an image only if the task remains unfinished and therefore returns no images after successful placement.

Programmatic composition reduces the LLM invocations needed for dependent operations, while selective feedback limits the outputs and images added to subsequent prompts. Persistent program state retains intermediate data outside the conversation. These mechanisms control new context rather than removing existing history: both agents run in the Codex CLI [20], which keeps the full conversation and compacts it, summarizing earlier turns, only when it approaches 272K tokens.

4 Experimental Results and Analysis

Our central question is whether a VLM agent can compose robot primitives to improve task completion while reducing token overhead through programmatic execution and selective feedback. We evaluate PyRUA-Lean

Table 1 The two agents’ interfaces: each primitive is an RPent tool for tool calling and a robo method with the same name and arguments for code, both running RPent’s implementation. Names pooled over the benchmarks.

	<i>Tool</i> (RPent)	<i>Code</i> (PyRUA-Lean)
<i>Primitives, the same in both</i>		
Arm motion	move_to, move_pose, move_delta, rotate_wrist, rotate_pitch	
Gripper	set_gripper, release, scripted_grasp	
VLA policy	pi0_pick, pi0_doubled, lingbot_act, rldx_skill, rldx_arm	
Mobile base	navigate_to, move_base	
Perception	segment, back_project, back_project_batch, sample_world_xyz, query_world_map	
<i>Observations</i>		
State, success flag	with every move, or view_env_state	robo.state(), robo.done
Camera images	three with every move	when the code calls robo.show
World coordinates	through the perception tools	the same, or robo.world_map() as one array

Table 2 Main results. Success rate on all task instances; LLM calls, prompt tokens and dollars at list prices per solved episode, on the task instances both agents solved (means). *Tool*: tool calling (RPent); *Code*: PyRUA-Lean (ours). Shaded: code’s gain, in points (Δ) or how many times fewer ($\times$). *All* pools the four benchmarks, where LIBERO-PRO’s long episodes dominate the token factor. RC365: RoboCasa365.

Benchmark	All task instances			Per solved episode, on the task instances both agents solved								
	Success rate (%)			LLM calls			Tokens			Cost (\$)		
	<i>Tool</i>	<i>Code</i>	Δ	<i>Tool</i>	<i>Code</i>	$\times$	<i>Tool</i>	<i>Code</i>	$\times$	<i>Tool</i>	<i>Code</i>	$\times$
LIBERO-PRO	83.0	94.0	+11.0	21.4	8.4	2.5 $\times$	1.23M	273k	4.5 $\times$	2.49	0.79	3.1 $\times$
RoboTwin 2.0	60.0	69.2	+9.2	12.2	8.9	1.4 $\times$	415k	245k	1.7 $\times$	1.00	0.66	1.5 $\times$
RC365 atomic	78.9	86.7	+7.8	10.4	5.6	1.8 $\times$	261k	171k	1.5 $\times$	0.62	0.54	1.2 $\times$
RC365 composite	34.4	39.4	+5.0	28.4	14.9	1.9 $\times$	1.22M	603k	2.0 $\times$	2.01	1.25	1.6 $\times$
All	63.1	71.7	+8.6	17.0	8.7	2.0$\times$	788k	276k	2.9$\times$	1.63	0.74	2.2$\times$

against a tool-calling baseline through three hypotheses (**H**):

- **H1**: PyRUA-Lean achieves higher task success than tool calling under equal LLM-call budgets.
- **H2**: PyRUA-Lean requires fewer LLM calls and input tokens to solve tasks.
- **H3**: PyRUA-Lean retains its token efficiency when VLA policies or operating guides are absent.

4.1 Experimental Setup

Benchmarks. We evaluate 40 tasks from LIBERO-PRO’s four perturbed suites [15, 34], 50 dual-arm tasks from RoboTwin 2.0 [6], and 50 tasks from RoboCasa365’s Target50 set [18]. Each task is evaluated at five environment seeds, with one episode per agent for each task–seed pair, yielding 700 paired task instances and 1,400 episodes. More implementation details can be found in Appendix F.

Agents. Both agents use GPT-6 Astra [21] at high reasoning effort through the Codex CLI, with the same robot stacks, primitive implementations, and frozen VLA policies: $\pi_{0.5}$ on LIBERO-PRO [23], LingBot-VLA on RoboTwin 2.0 [30], and RLDX-1 on RoboCasa365 [12]. Both use SAM 3 for segmentation [5]. The baseline invokes RPent’s primitives through tool calls; PyRUA-Lean composes them through its Python interface. Both receive the same operating guides where available; RoboCasa365 has none. Both agents operate without cross-episode memory.

Metrics. Success rate is measured over all task instances using each benchmark’s success check, with budgets of 40 LLM calls per episode, or 100 for RoboCasa365 composite tasks, and two hours. The LLM-call budgets constrain model invocations rather than the number of primitive executions. We compare LLM calls, cumulative input tokens, and estimated monetary cost on instances solved by both agents, reporting whole-episode means and ratios of aggregate totals. Counts include post-success calls and cached input tokens. Cost-accounting details and all-episode costs are reported in Appendix E.

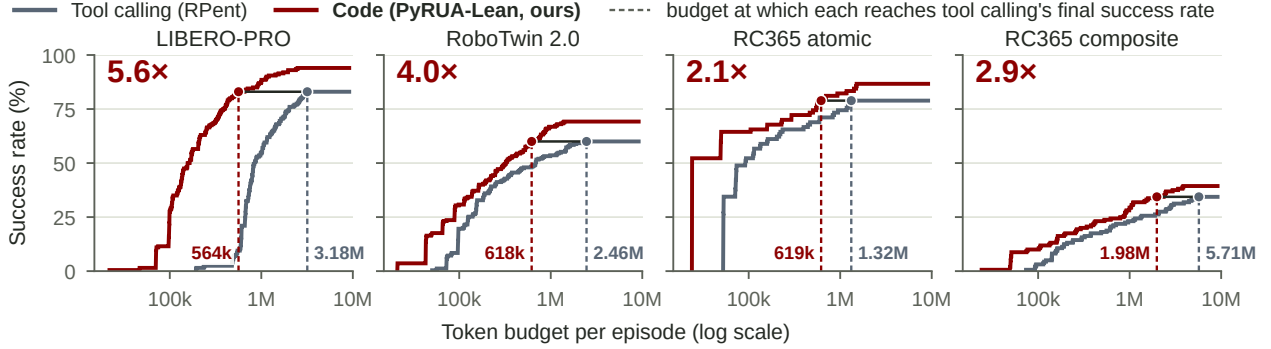


Figure 3 Success rate if every episode were stopped once its token consumption reached the budget on the horizontal axis. Dashed lines mark the budget at which each agent first reaches tool calling’s final success rate, with its value on the axis; the large number is their ratio. RC365: RoboCasa365.

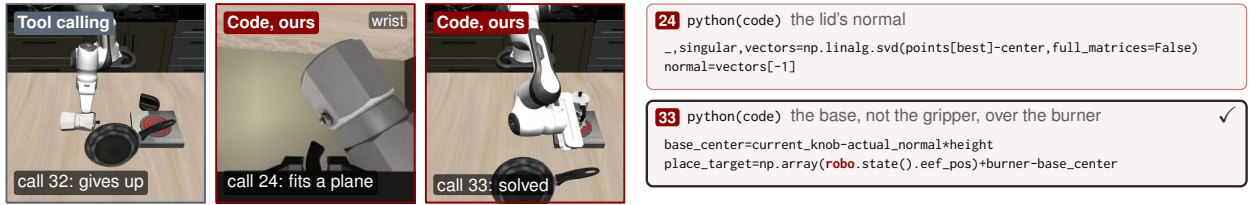


Figure 4 Only code solved this LIBERO-PRO instance (“turn on the stove and put the moka pot on it”): both agents dropped the pot, and the code agent fitted a plane to the lid, turned the pot upright and set its base over the burner (Appendix D).

4.2 Task Success and Inference Cost

Task success improves across all benchmark groups. Table 2 shows that PyRUA-Lean increases overall success from 63.1% to 71.7%, an improvement of 8.6 percentage points, or approximately 14% relative. The gains are 11.0 points on LIBERO-PRO, 9.2 on RoboTwin 2.0, 7.8 on RoboCasa365 atomic tasks, and 5.0 on composite tasks. PyRUA-Lean alone solves 96 instances, whereas tool calling alone solves 36. These results support Hypothesis 1 within the evaluated setting: PyRUA-Lean completes more tasks under the same LLM-call budget. Figure 4 illustrates a recovery example in which PyRUA-Lean uses geometric computation to reposition a dropped object and complete the task. Appendix D provides further analysis of instances solved by only one agent.

Completing the same tasks requires fewer calls and tokens. On jointly solved instances, PyRUA-Lean reduces mean LLM calls from 17.0 to 8.7 and input tokens from 788k to 276k, corresponding to reductions of 49% and 65%. Estimated cost decreases from \$1.63 to \$0.74 per episode. The smaller monetary reduction reflects, in part, caching discounts on the baseline’s repeatedly processed history. These findings support Hypothesis 2 and show that the higher overall success rate is accompanied by lower inference overhead on shared successes. The pooled token reduction is influenced strongly by LIBERO-PRO’s longer episodes; per-benchmark results are therefore also reported.

Figure 3 complements this comparison with retrospective token-budget cutoffs on recorded trajectories. On LIBERO-PRO, PyRUA-Lean reaches the baseline’s final success rate with 564k tokens per episode, compared with 3.18M for tool calling. This connects success and efficiency beyond the jointly solved subset, although the agents were not rerun with these token budgets specified beforehand.

RoboDojo pilot. We additionally compare RPent-adapted and PyRUA-Lean-adapted on three RoboDojo [7] tasks: *cover blocks*, *press by number*, and *stack blocks by language*—using the same GPT-6 Astra planner with identical task-agnostic primitives. PyRUA-Lean achieves 26.7% success (8/30) versus 3.4% (1/29 scored; one infrastructure failure excluded); both fail button pressing. Total input and output tokens across all attempts per success decrease from 11.56M to 1.07M (90.7%), while total-token usage on 29 matched scored pairs falls

by 25.3%.

4.3 Analysis of LLM Calls and Observation Feedback

Fewer VLM invocations account for most of the token reduction. Figure 5 decomposes the token ratio into the LLM-call ratio and the ratio of average input tokens per call. On LIBERO-PRO, the 4.48 $\times$ token ratio comprises 2.55 $\times$ fewer calls and 1.76 $\times$ fewer tokens per call. On RoboCasa365 atomic tasks, PyRUA-Lean’s calls are larger on average, yet fewer invocations still reduce total token usage. Thus, Hypothesis 2 is supported primarily by reducing how often the VLM is invoked, rather than uniformly making each prompt smaller.

Execution traces illustrate how programmatic composition enables this reduction. A cell can execute dependent motions, check their outcomes, and retry a primitive before returning control to the VLM. In the bowl-placement example, the agent also computes placement geometry directly from observations. These behaviors address the central research question by moving intermediate coordination and computation into the execution runtime. The token decomposition is an accounting analysis, however, and does not independently isolate the causal contribution of each interface feature.

On-demand images alone do not reproduce the gains. On LIBERO-PRO, we additionally modify the tool-calling baseline to return images only on request. As shown in Table 8, success decreases from 83.0% to 68.5%. On the 131 instances solved by all three configurations, the modified baseline makes more calls and consumes approximately the same total input tokens as the original baseline. This suggests that reducing automatic visual feedback alone is insufficient: observation requests must be considered together with action execution and replanning. The experiment does not separately quantify the contribution of selective feedback within PyRUA-Lean.

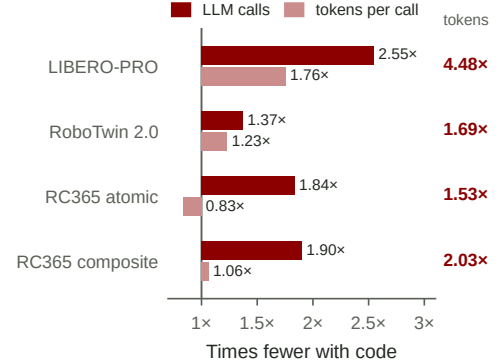


Figure 5 Token-savings decomposition on jointly solved instances. Tool-calling-to-PyRUA-Lean ratios quantify reductions in LLM calls (solid) and tokens per call (light); their product gives the total token-reduction factor (right). Ratios below 1 $\times$ indicate increased usage by PyRUA-Lean.

4.4 Ablation Studies

Token savings persist without VLA policies or guides. We remove VLA policies, operating guides, or both from each agent on the same task instances; RoboCasa365 has no guides, so only VLA policies are ablated. Across these settings, PyRUA-Lean maintains higher overall success and lower input-token usage on jointly solved instances (Table 3), supporting Hypothesis 3. Without VLA policies, it achieves 68.8% success on RoboTwin 2.0 versus 42.8% for tool calling, with traces showing classical primitives composed into approach, gripper control, and state checks.

Guidance has interface-dependent effects. On RoboTwin 2.0 without VLA policies, removing guides increases tool-calling success from 42.8% to 60.4%, suggesting that guidance is not uniformly beneficial. Jointly solved subsets also vary across settings, so their token costs do not isolate component effects on a fixed task subset.

4.5 Failure Cases and Limitations

Improved budget use explains some, but not all, additional successes. Of the 96 instances solved only by PyRUA-Lean, the baseline exhausts its LLM-call budget in 52 and ends unsuccessfully before exhausting it in 44. The former cases are consistent with programmatic composition enabling more execution within the budget. The latter include examples of geometric reasoning and scripted recovery, but also cases where a VLA succeeds in one run and fails in the other. The success difference therefore cannot be attributed entirely to better recovery logic.

Efficiency gains depend on task structure and failures. Gains are smallest on RoboCasa365 atomic tasks, where a VLA policy often completes most of the task and the initial prompt accounts for much of the input. PyRUA-Lean’s larger API description can offset part of the benefit from fewer calls. Including failed episodes,

Table 3 Ablations: the VLA policy, the operating guides or both removed from *both* agents, on every task instance; each group’s first row is Table 2’s. RoboCasa365 has no operating guides (–).

VLA	Guides	Success rate (%)			LLM calls			Tokens			Cost (\$)		
		<i>Tool</i>	<i>Code</i>	Δ	<i>Tool</i>	<i>Code</i>	$\times$	<i>Tool</i>	<i>Code</i>	$\times$	<i>Tool</i>	<i>Code</i>	$\times$
LIBERO-PRO													
✓	✓	83.0	94.0	+11.0	21.4	8.4	2.5×	1.23M	273k	4.5×	2.49	0.79	3.1×
✓	✗	84.0	88.0	+4.0	19.0	9.3	2.0×	901k	286k	3.1×	1.89	0.71	2.6×
✗	✓	73.0	80.5	+7.5	22.9	10.9	2.1×	1.34M	355k	3.8×	2.60	0.86	3.0×
✗	✗	74.0	82.0	+8.0	20.0	9.8	2.0×	943k	290k	3.2×	2.00	0.75	2.7×
RoboTwin 2.0													
✓	✓	60.0	69.2	+9.2	12.2	8.9	1.4×	415k	245k	1.7×	1.00	0.66	1.5×
✓	✗	64.0	66.8	+2.8	11.6	9.1	1.3×	392k	251k	1.6×	0.91	0.64	1.4×
✗	✓	42.8	68.8	+26.0	23.1	11.5	2.0×	1.03M	316k	3.2×	1.94	0.85	2.3×
✗	✗	60.4	66.4	+6.0	16.9	12.0	1.4×	620k	329k	1.9×	1.29	0.88	1.5×
RoboCasa365 atomic													
✓	–	78.9	86.7	+7.8	10.4	5.6	1.8×	261k	171k	1.5×	0.62	0.54	1.2×
✗	–	45.6	65.6	+20.0	26.5	18.7	1.4×	760k	627k	1.2×	1.48	1.37	1.1×
RoboCasa365 composite													
✓	–	34.4	39.4	+5.0	28.4	14.9	1.9×	1.22M	603k	2.0×	2.01	1.25	1.6×
✗	–	7.5	20.0	+12.5	58.5	31.1	1.9×	2.58M	1.14M	2.3×	5.50	3.08	1.8×

the estimated VLM inference cost of tool calling on RoboCasa365 atomic tasks is approximately 1.1 times that of PyRUA-Lean (Appendix E). Efficiency on jointly solved instances should therefore be distinguished from the cost of all attempts.

5 Limitations and Future Work

Our evaluation is limited to GPT-6 Astra, RPent’s robot stacks and primitive libraries, and simulation, with one run per agent per task instance. Generalization to other planners and primitive configurations, variability across repeated runs, and physical-robot performance remain untested. The comparison evaluates the complete interfaces without fully isolating the effects of primitive composition, persistent state, and selective feedback. Future work will address these limitations, measure execution latency and recovery on physical robots, and examine how primitive granularity and cross-episode reuse of validated routines affect success, inference cost, and adaptability.

6 Conclusion

We presented PyRUA-Lean, an interactive code-execution interface that composes existing robot primitives, including VLA policies, and controls which execution feedback enters the LLM context. Across 700 simulated task instances, PyRUA-Lean outperforms a tool-calling baseline using the same GPT-6 Astra planner and underlying primitives under equal LLM-call budgets, increasing overall success from 63.1% to 71.7%. On instances solved by both agents, it uses 49% fewer LLM calls and 65% fewer input tokens. Fewer LLM invocations account for most of the token savings, with smaller gains on VLA-dominated tasks. These results demonstrate the value of programmatic primitive composition and motivate further study of feedback management in robot agents.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.
- [2] Anthropic. Code execution with MCP: Building more efficient agents. Anthropic Engineering Blog, <https://www.anthropic.com/engineering/code-execution-with-mcp>, 2025.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, et al. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- [5] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, et al. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.
- [6] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [7] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, et al. Robodojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.
- [8] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, et al. Show-Harness: Just a VLM agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026.
- [9] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, et al. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. In *International Conference on Machine Learning (ICML)*, 2026.
- [10] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2022.
- [11] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.
- [12] Dongyoung Kim, Huiwon Jang, Myungkyu Koo, Suhyeok Jang, Taeyoung Kim, Beomjun Kim, Byungjun Yoon, Changsung Jang, et al. RLDX-1 technical report. *arXiv preprint arXiv:2605.03269*, 2026.
- [13] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.
- [14] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [15] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [16] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, et al. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.
- [17] Dhia Naouali, Minghan Wu, Claudia Wong, Abhinav Puthran, and Omar G. Younis. VLCP: Vision language control policy closed-loop code replanning for robot manipulation. *arXiv preprint arXiv:2608.16978*, 2026.

- [18] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.
- [19] Noematrix Team. RoboRSI: Stable, efficient, and reusable robot self-evolution in complex real-world environments. Blog post, <https://lab.noematrix.ai/blog/2-roborsi/>, 2026.
- [20] OpenAI. Codex CLI. <https://github.com/openai/codex>, 2025.
- [21] OpenAI. GPT-6 Astra: A new generation of intelligence. <https://openai.com/index/gpt-6-astra/>, 2026.
- [22] OpenAI. Pricing. OpenAI API documentation, <https://developers.openai.com/api/docs/pricing>, 2026.
- [23] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [25] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, et al. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [26] Jiayi Su, Yixin Zheng, Mi Yan, Li Yi, Zhizheng Zhang, and He Wang. GPT 6 Astra as an embodied policy. Technical report, <https://anonymous-report-421.github.io/public-website/>, 2026.
- [27] Kenton Varda and Sunil Pai. Code Mode: the better way to use MCP. Cloudflare Blog, <https://blog.cloudflare.com/code-mode/>, 2025.
- [28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [29] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024.
- [30] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, et al. A pragmatic VLA foundation model. *arXiv preprint arXiv:2601.18692*, 2026.
- [31] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [33] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaying Qiu, et al. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.
- [34] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

Fewer Tokens, Better Action: GPT-6 Astra Robot Agents with 14% Higher Success Rate but 65% Fewer Tokens

Appendix

A The episode of Figure 2, call by call

Figure 2 shows one stretch of an episode. Figure 6 and Table 4 follow the whole episode with each agent: of the instances both agents solved, it is the one closest to the median cost for both (LIBERO-PRO, spatial swap, task 7, seed 2). The tool-calling agent reads the guides, localizes, and then moves one step per call, getting three camera images back after every move; it solves the task in call 17, after 33 images in all. The code agent localizes both objects in its first cell, approaches and grasps in its second, closes the gripper and lifts the bowl in its third, checks the grasp in its fourth, and carries and places the bowl in its fifth and sixth; it solves the task in call 6 and asks for 5 images in all.

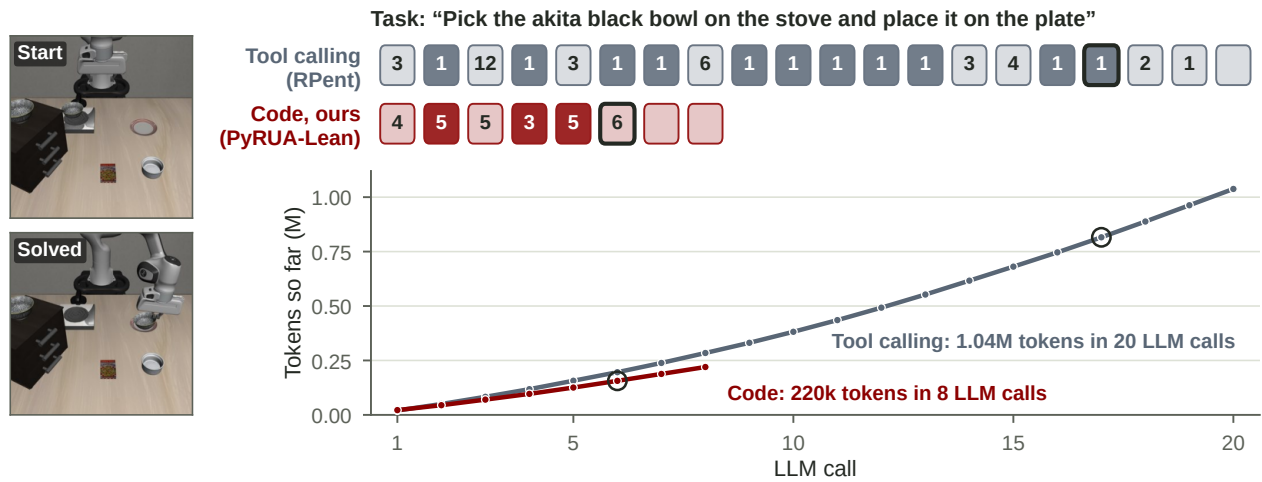


Figure 6 The episode of Figure 2, whole. *Top:* each box is one LLM call; the number is how many tools (tool calling) or primitives (code) it called, dark boxes returned camera images, and the outlined box solved the task. *Bottom:* token consumption so far; circles mark the solving call, reached after 816k tokens with tool calling and 156k with code. *Left:* the agent-view camera at the start and when the task was solved.

Cells 1–6 of the code agent are reproduced exactly as generated by the model.

LLM call 1

```
print(robo.state())
bowl = robo.segment('the black patterned bowl on the stove')
plate = robo.segment('the white plate with red rings')
print('bowl', bowl)
print('plate', plate)
print('table', robo.back_project(630, 610))
```

LLM call 2

```
assert bowl.found and plate.found
approach = robo.move_to([bowl.world_xyz[0], bowl.world_xyz[1], bowl.world_xyz[2]+0.19], gripper=-1)
print('approach', approach)
if approach.reached:
    pick = robo.pi0_pick('pick up the black bowl on the stove', max_chunks=20)
    print('pick', pick)
print(robo.state())
robo.show('agentview')
robo.show('wrist')
```

LLM call 3

```
print(robo.set_gripper(+1, steps=8))
held_state = robo.state()
```

Table 4 The two episodes of Figure 6, LLM call by LLM call: what each call ran, and the token consumption so far. A check mark is the call that solved the task. Cached input is included, as everywhere.

Call	Tool calling (RPent): tools called	Tokens	Code (PyRUA-Lean): what the cell ran	Tokens
1	read_text_file ×3	20k	4 primitive calls: back_project, segment, view_env_state	22k
2	view_env_state; 3 images back	50k	5 primitive calls: move_to, pi0_pick, view_env_state; shows agentview and wrist	44k
3	back_project ×12	83k	5 primitive calls: move_to, segment, set_gripper, view_env_state	70k
4	move_to; 3 images back	118k	3 primitive calls: segment; shows agentview and wrist	96k
5	back_project ×3	157k	5 primitive calls: move_to, segment, view_env_state; shows agentview	126k
6	move_to; 3 images back	196k	6 primitive calls: move_to, release, segment, view_env_state ✓	156k
7	move_to; 3 images back	239k	finish	188k
8	back_project ×6	284k	final message	220k
9	pi0_pick; 3 images back	331k		
10	set_gripper; 3 images back	382k		
11	move_to; 3 images back	435k		
12	move_to; 3 images back	492k		
13	move_to; 3 images back	553k		
14	back_project ×3	616k		
15	back_project ×4	681k		
16	move_to; 3 images back	746k		
17	release; 3 images back ✓	816k		
18	write_text_file ×2	888k		
19	finish	963k		
20	final message	1.04M		

```
print('lift', robo.move_to([held_state.eef_pos[0], held_state.eef_pos[1], 1.18], gripper=+1, step_clip=0.012))
held_bowl = robo.segment('the black patterned bowl held by the gripper')
print('held bowl', held_bowl)
print(robo.state())
```

LLM call 4

```
robo.show('agentview')
robo.show('wrist')
print('bowl now', robo.segment('the black patterned bowl on the stove'))
```

LLM call 5

```
held_bowl = robo.segment(point=(267, 339))
print('held localization', held_bowl)
assert held_bowl.found and held_bowl.world_xyz[2] > 1.05
carry_state = robo.state()
grip_offset = np.array(carry_state.eef_pos[:2]) - np.array(held_bowl.world_xyz[:2])
place_xy = np.array(plate.world_xyz[:2]) + grip_offset
print('grip offset', grip_offset, 'place xy', place_xy)
mid_xy = (np.array(carry_state.eef_pos[:2]) + place_xy)/2
for waypoint_xy in [mid_xy, place_xy]:
    move = robo.move_to([*waypoint_xy, carry_state.eef_pos[2]], gripper=+1, step_clip=0.012)
    print('carry', move)
    if not move.reached or move.terminated:
        break
robo.show('agentview')
```

LLM call 6

```
held_at_plate = robo.segment(point=(314, 795))
print('bowl above plate', held_at_plate)
assert held_at_plate.found
correction = np.array(plate.world_xyz[:2]) - np.array(held_at_plate.world_xyz[:2])
place_xy = np.array(robo.state().eef_pos[:2]) + correction
surface_map = robo.world_map()
bowl_patch = surface_map[290:365, 740:850]
bowl_heights = bowl_patch[:, :, 2]
bowl_heights = bowl_heights[(bowl_heights>1.04) & (bowl_heights<1.17)]
bottom_z = float(np.quantile(bowl_heights, 0.02))
placement_z = plate.world_xyz[2] + robo.state().eef_pos[2] - bottom_z + 0.007
print('correction', correction, 'bottom', bottom_z, 'placement z', placement_z)
assert np.linalg.norm(correction)<0.07 and 0.95<placement_z<1.05
lower = robo.move_to([*place_xy, placement_z], gripper=+1, step_clip=0.012, tol=0.005, max_steps=100)
print('lower', lower)
if lower.reached and not robo.done:
```

```
print('release', robo.release())
print('done', robo.done)
if not robo.done:
    robo.show('agentview')
```

B More episodes

Table 5 and Figure 7 add five tasks per benchmark that both agents solved: on each benchmark, the task instances whose token factor (tool calling’s token consumption over code’s, whole episodes) is closest to the benchmark’s median, at most one per task and none of Figure 6’s task.

Table 5 The episodes of Figure 7: LLM calls and token consumption of each whole episode, with tool calling (*Tool*) and with code (*Code*); shaded: how many times fewer tokens code consumed.

Benchmark	Task	Seed	LLM calls		Tokens		×
			<i>Tool</i>	<i>Code</i>	<i>Tool</i>	<i>Code</i>	
LIBERO-PRO	object swap 0: Pick the alphabet soup and place it in the basket	1	21	8	1.09M	219k	5.0×
	spatial swap 5: Pick the akita black bowl on the ramekin and place it on the plate	1	18	7	918k	188k	4.9×
	goal swap 4: Put the bowl on the top of the drawer	3	19	7	991k	199k	5.0×
	spatial swap 9: Pick the akita black bowl on the wooden cabinet and place it on the plate	3	24	9	1.43M	288k	5.0×
	object swap 3: Pick the bbq sauce and place it in the basket	1	17	6	814k	161k	5.0×
RoboTwin 2.0	place container plate	100000	7	4	151k	89k	1.7×
	open laptop	100008	7	4	149k	88k	1.7×
	place empty cup	100001	7	4	150k	89k	1.7×
	rotate qr code	100001	7	4	151k	89k	1.7×
	move pill bottle pad	100002	7	4	152k	91k	1.7×
RoboCasa365 atomic	CloseToasterOvenDoor	1	5	3	91k	74k	1.2×
	PickPlaceToasterToCounter	4	5	3	92k	75k	1.2×
	CoffeeSetupMug	5	5	3	92k	75k	1.2×
	TurnOnSinkFaucet	4	5	3	91k	74k	1.2×
	TurnOnElectricKettle	4	5	3	91k	75k	1.2×
RoboCasa365 composite	ScrubCuttingBoard	3	8	4	159k	101k	1.6×
	WashLettuce	1	8	4	162k	100k	1.6×
	StackBowlsCabinet	1	6	3	117k	78k	1.5×
	CuttingToolSelection	2	18	9	438k	253k	1.7×
	LoadDishwasher	5	15	7	361k	193k	1.9×

C What the code agent writes

We read the code agent’s cells on every benchmark, in solved and failed episodes, with and without the VLA policy, and then counted what they do over all 700 episodes of the main comparison, 11,145 cells, each episode cut at its budget (Table 6).

A cell acts more than a tool call does. A code cell runs 2.1 acting primitives on average (moves, grasps, gripper commands and VLA calls), and 45% of cells run two or more; an LLM call of tool calling runs 0.65. Of the tool-calling calls that call a tool, 89% call exactly one; of the 1,907 that call several, 1,441 batch back-projections and 292 read or write files, and only 45 of all 18,585 run two acting tools together. Five habits of the code agent make up the difference, and none of them fits in one tool call: the first two fold steps that tool calling spreads over several LLM calls into one, the next two build what the tool list does not offer, and the last decides when to look.

Guarded chains (19% of cells, 45% on LIBERO-PRO). A cell runs several primitives in a row and makes each depend on the result of the one before (Listing 1): it moves above the bowl, grasps with the VLA policy only

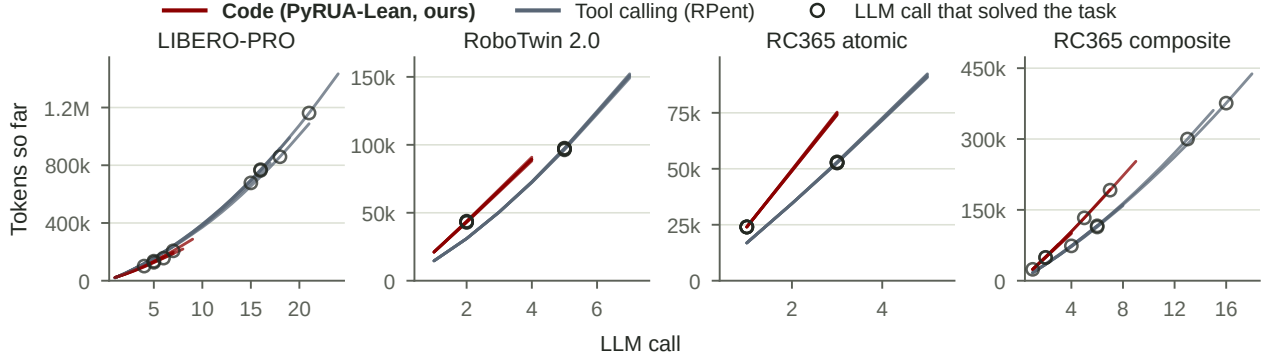


Figure 7 Token consumption so far of the episodes in Table 5, five per benchmark, LLM call by LLM call; circles mark the solving call. On RoboTwin 2.0 and RoboCasa365’s atomic tasks the median episodes of both agents are nearly identical from task to task, so their lines overlap.

if the move arrived and the task is not done, and looks only if it is still not done. With tool calling, each link of such a chain is an LLM call.

Retries (8% of cells, 18% on RoboCasa365’s atomic tasks). Where a VLA policy may stop short of the goal, the cell runs it again until the task reports success (Listing 2). With tool calling, every retry is a round trip that brings back three camera images.

Perception of its own (30% of cells compute with NumPy, for perception or geometry). A cell can read a camera’s image and point cloud as arrays and locate objects itself: a color mask for a block, the points above a height for its top, the principal axis of those points for its orientation, and from that the yaw of the grasp (Listing 3). A tool-calling agent gets what its tools compute (images, the 3D point at a pixel it names, statistics of a box it names), and must work out anything else in its reasoning.

Skills of its own (2% of cells define a helper function, 7% call one that an earlier cell defined). In Listing 4, the agent writes a descent that moves down in 8 mm steps and stops as soon as a step does not reach its target, stops going down, or drifts sideways, and it reuses the function in its next cell. Helpers like this are how the code agent grasps without a VLA policy (Section 4.4).

Looking when needed (20% of cells). The code agent looks often, 84% of its cells ask for a camera image, but it chooses when and through which camera; many cells look only if the task is not yet done, where tool calling gets three images after every move whether it needs them or not.

And single steps (35% of cells, 54% on RoboTwin 2.0). Many cells take one step and look, like a tool call. On RoboTwin 2.0 the agent often runs the VLA policy one chunk at a time and checks the result after each; code saves the fewest LLM calls there (Table 2).

```

assert bowl.found and plate.found
bowl_xyz = np.array(bowl.world_xyz[:2])
plate_xyz = np.array(plate.world_xyz)
table_z = robo.back_project(837,660).world_xyz[2]
prepose = [float(bowl_xyz[0]),float(bowl_xyz[1]),table_z+0.22]
approach = robo.move_to(prepose,gripper=robo.OPEN)
print('APPROACH',approach)
if approach.reached and not robo.done:
    picked = robo.pi0_pick('pick up the black bowl between the plate and the ramekin', max_chunks=20)
    print('PICK',picked)
print('STATE',robo.state())
if not robo.done:
    robo.show('agentview')
    robo.show('wrist')

```

Listing 1 A guarded chain: cell 2 of the code agent in LIBERO-PRO, spatial swap, task 0, seed 0, exactly as the model wrote it.

Table 6 What the code agent’s cells do, over all 700 episodes of the main comparison (each cut at its budget), against what an LLM call of tool calling does on the same task instances. Acting primitives are moves, grasps, gripper commands and VLA calls. RC365: RoboCasa365.

	All	LIBERO-PRO	RoboTwin 2.0	RC365 atomic	RC365 composite
Acting primitives per code cell	2.1	2.0	1.9	2.1	2.4
Acting tools per LLM call of tool calling	0.65	0.60	0.69	0.56	0.67
<i>Share of code cells that</i>					
run two or more acting primitives	45%	61%	24%	50%	56%
take a single step and look	35%	14%	54%	33%	28%
make an action depend on an earlier result	19%	45%	12%	20%	14%
loop over motions	14%	6%	12%	22%	17%
retry a VLA policy	8%	0%	0%	18%	17%
compute with NumPy	30%	25%	31%	29%	30%
reuse a variable from an earlier cell	19%	33%	18%	10%	14%
call a helper function from an earlier cell	7%	7%	6%	2%	9%
ask for a camera image	84%	82%	78%	90%	90%
look only if the task is not done	20%	19%	8%	58%	24%

```

print(robo.task)
print(robo.success_criteria())
for attempt in range(3):
    result = robo.rldx_arm()
    print(result)
    print(robo.state())
    if robo.done or result.status != 'cap':
        break
if not robo.done:
    robo.show('agentview')
    robo.show('wrist')

```

Listing 2 A retry: cell 1 of the code agent in RoboCasa365 atomic, CloseToasterOvenDoor, seed 1, exactly as the model wrote it.

D Instances only one agent solved

Code alone solves 96 task instances of the main comparison, and tool calling alone 36. In 52 of the 96, tool calling ran out of LLM calls; in the other 44, it ended the episode itself and reported failure. We read both episodes of each of those 44. In about a quarter, the code agent only called the VLA policy, and the policy succeeded in its episode but not in tool calling’s. In most of the others, both agents met the same setback, most often a VLA grasp that missed or an object that fell. Tool calling then asked the policy again, or stopped with most of its budget left; the code agent closed the gripper itself, scripted the recovery from the same primitives, checked each step, and in some episodes computed from the camera’s point cloud what no tool returns. Figures 8 and 9 follow two instances, one where tool calling ended the episode itself and one where it ran out of calls.

E Cost accounting

Episode-level accounting. We removed memory-, recipe-, and audit-related instructions from the prompts and operating guides, while retaining the baseline’s original tool descriptions. These descriptions may still prompt the baseline to save episode-local recipe and audit files after success. Such files are stored in the episode’s own output directory, and inspection of the evaluation traces found no cross-episode file reads. Any associated post-success calls are included in the reported whole-episode metrics.

Prices. Every LLM call of every episode is billed from its own token usage at GPT-6 Astra’s list prices as of September 2026 [22]: \$10 per million input tokens, \$1 for cached input, \$12.50 for cache writes and \$50 for output, reasoning included. A call above 272K prompt tokens would cost twice as much for input and 1.5 times as much for output; none reached it.

Calls, not seconds. We count LLM calls rather than time episodes, because the latency of an LLM call on a shared model gateway varies too much from call to call to compare.

```

mask=red&(rows<92)&(cols>98)&(cols<165)&(world[... ,2]>0.745)
points=world[mask]
upper=points[points[:,2]>0.794]
mean_xy=np.mean(upper[:,2],axis=0)
eigenvalues,eigenvectors=np.linalg.eigh(np.cov(upper[:,2].T))
long_axis=eigenvectors[:,1]
if long_axis[1]<0: long_axis=-long_axis
short_axis=np.array([long_axis[1],-long_axis[0]])
projection_long=upper[:,2]@long_axis
projection_short=upper[:,2]@short_axis
block_center=long_axis*((np.min(projection_long)+np.max(projection_long))/2)+short_axis*((np.min(projection_short)+np.max(projection_short))/2)
print('flat centre',block_center,'long axis',long_axis,'extents',np.ptp(projection_long),np.ptp(projection_short))
left_grasp_xy=block_center+0.045*long_axis
right_grasp_xy=block_center-0.05*long_axis
yaw=math.atan2(long_axis[1],long_axis[0])
q_down=np.array([math.cos(yaw/2)/math.sqrt(2),-math.sin(yaw/2)/math.sqrt(2),math.cos(yaw/2)/math.sqrt(2),math.sin(yaw/2)/math.sqrt(2)])
print('grasp',left_grasp_xy,'receive',right_grasp_xy,'quat',q_down)
safe=np.array(robo.state().left.eef_pos);safe[2]=1.03
if checked_move('left',safe,quat=robo.state().left.eef_quat,substeps=20):
    waypoint=np.array([left_grasp_xy[0],0.015,1.03])
    if checked_move('left',waypoint,quat=q_down,substeps=25):
        print(robo.move_to('left',[left_grasp_xy[0],left_grasp_xy[1],0.97],quat=q_down,substeps=25))
print(robo.state().left)
robo.show('head')

```

Listing 3 Perception of its own: cell 5 of the code agent in RoboTwin 2.0 without the VLA policy, handover block, seed 100003, exactly as the model wrote it. `checked_move`, a helper it defined in an earlier cell, moves an arm and reports whether the move arrived.

```

def lower_guarded(arm_name, target_tcp_z):
    initial=robo.state().arm(arm_name)
    target=np.array(initial.eef_pos)
    final_eef_z=target_tcp_z-0.12*initial.approach[2]
    while target[2]>final_eef_z+0.001 and not robo.done:
        before=robo.state().arm(arm_name)
        target[2]=max(final_eef_z,target[2]-0.008)
        result=robo.move_to(arm_name,target,substeps=8)
        after=robo.state().arm(arm_name)
        print('descent',result.reached,tuple(round(v,4) for v in after.tcp_pos))
        if result.terminated or not result.planned or not result.reached or after.eef_pos[2]>=before.eef_pos[2]-0.001 or np.linalg.norm(np.array(
            (after.eef_pos[:2])-target[:2]))>0.01:
            print('stop',result)
            break
    lower_guarded('right',0.800)
    robo.show('right_wrist')
    robo.show('head')

```

Listing 4 A skill of its own: cell 7 of the code agent in RoboTwin 2.0, place object stand, seed 100002, exactly as the model wrote it.

All-episode accounting. The efficiency metrics in Table 2 are computed on jointly solved instances, while success rates use all task instances. Aggregating over all episodes, including failures, the tool-calling-to-PyRUA-Lean input-token ratios are $3.1\times$ on LIBERO-PRO, $1.8\times$ on RoboTwin 2.0, and $1.6\times$ on RoboCasa365 composite tasks. On RoboCasa365 atomic tasks, estimated inference costs are similar, with a tool-calling-to-PyRUA-Lean cost ratio of $1.1\times$. Failed episodes on these tasks tend to approach the 40-call limit for both agents, while PyRUA-Lean uses more input tokens per call.

First prompts and output. The median first prompt, which holds the system prompt, the guides and the tool list or API reference, is 20k for tool calling and 22k for code on LIBERO-PRO, 15k and 21k on RoboTwin 2.0, and 17k and 24k on RoboCasa365. Output tokens, reasoning included, are at most 17% of either agent’s bill on any benchmark.

F Benchmarks and task instances

Table 7 lists what the main comparison ran: 700 task instances and 1,400 episodes. LIBERO-PRO runs every task at seeds 0–4, RoboTwin 2.0 at seeds 100000–100004 and RoboCasa365 at seeds 1–5. Where RoboTwin 2.0 cannot build a task at a seed, we use the next seed it can build, and an episode that ended because its simulator or VLA server stopped answering was run again; one RoboCasa365 instance (CoffeeSetupMug, seed

4) renders only noise on every camera, so that task runs at seed 6 instead.

Table 7 The task instances of the main comparison. Each instance runs once with each agent.

Benchmark	Tasks	Seeds per task	Task instances	LLM calls per episode
LIBERO-PRO	40	5	200	40
RoboTwin 2.0	50	5	250	40
RC365 atomic	18	5	90	40
RC365 composite	32	5	160	100
All	140		700	

G Ablation settings

The ablations of Table 3 run on every task instance of the main comparison, at its seeds. Every setting of a group uses the same task instances, and its first setting re-uses the episodes of the main comparison.

Tool calling with images on demand. On every LIBERO-PRO task instance, the tool-calling agent ran once more with one change: its tools return their text but no camera images, which it sees only when it calls RPent’s camera tool (three views per call), and its prompt no longer promises images after every tool call. Code needs no such run, since a PyRUA-Lean cell returns images only when it asks for them; the Code column in Table 8 reuses the PyRUA-Lean episodes from the main comparison.

Table 8 Tool calling with camera images after every move (the LIBERO-PRO baseline of Table 3) and only on demand, against code, on all 200 task instances of LIBERO-PRO. *On the 131 instances all three solved (means).

	<i>Tool</i> , images after every move	<i>Tool</i> , images on demand	<i>Code</i>
Success rate (%), all task instances	83.0	68.5	94.0
Failures: gave up / call budget	9 / 25	17 / 46	5 / 7
LLM calls per solved episode*	19.9	22.8	7.4
Prompt tokens per solved episode*	1.07M	1.07M	222k

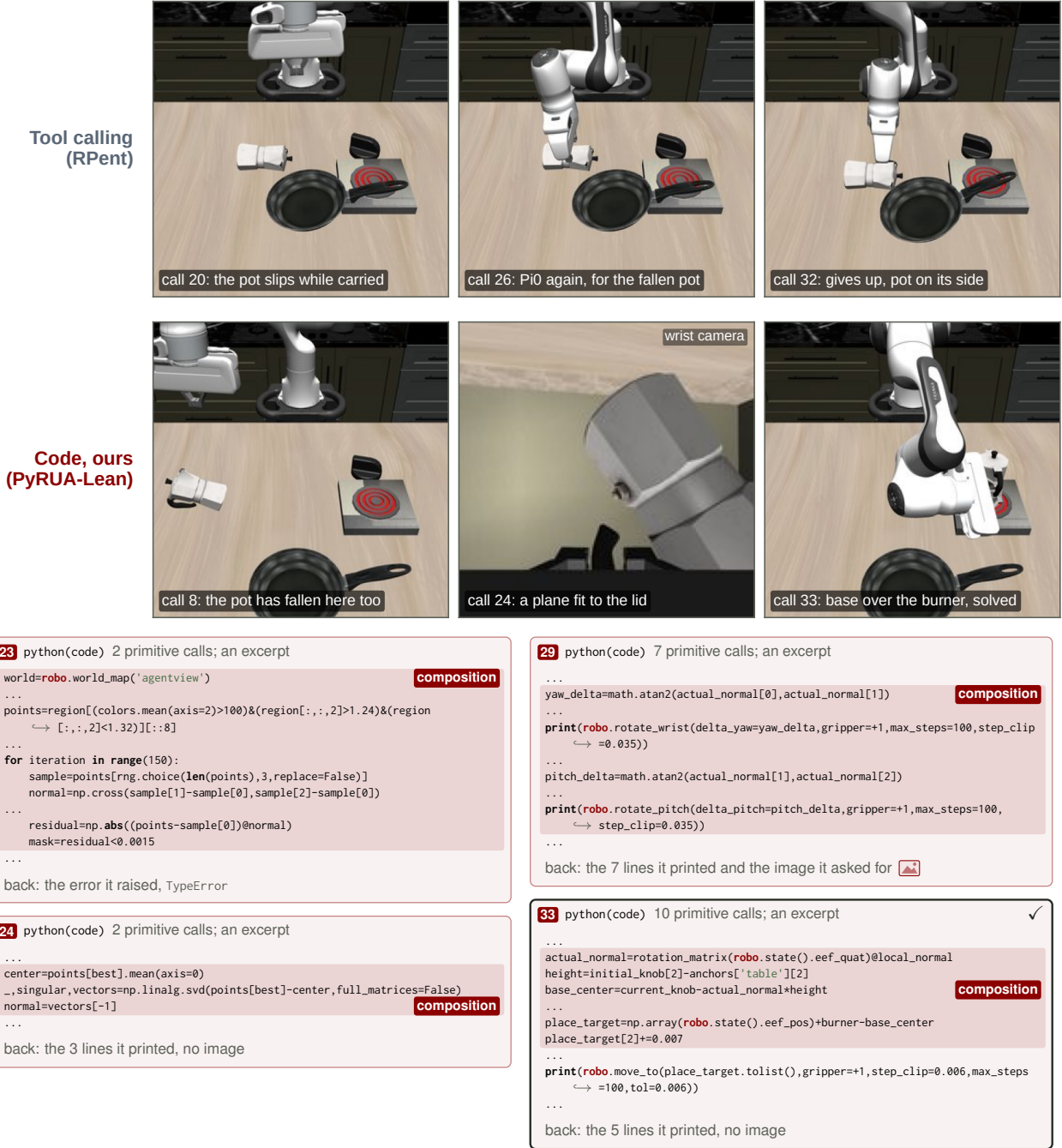


Figure 8 An instance only code solved: LIBERO-PRO, “turn on the stove and put the moka pot on it” (10 swap, task 2, seed 3). Both agents turned the stove on and dropped the pot. Tool calling asked Pi0 to grasp the fallen pot four times and ended the episode; the code agent grasped it itself, fitted a plane to the points of its lid to find how it leans, turned it upright, and set its base, not the gripper, over the burner. *Top*: the recorded camera views after the LLM calls named. *Bottom*: the code agent’s cells, excerpts exactly as the model wrote them; “...” marks lines left out. The plane fit in call 23 raised an error (NumPy’s linear algebra refuses half-precision arrays); call 24 cast the points and ran it again.

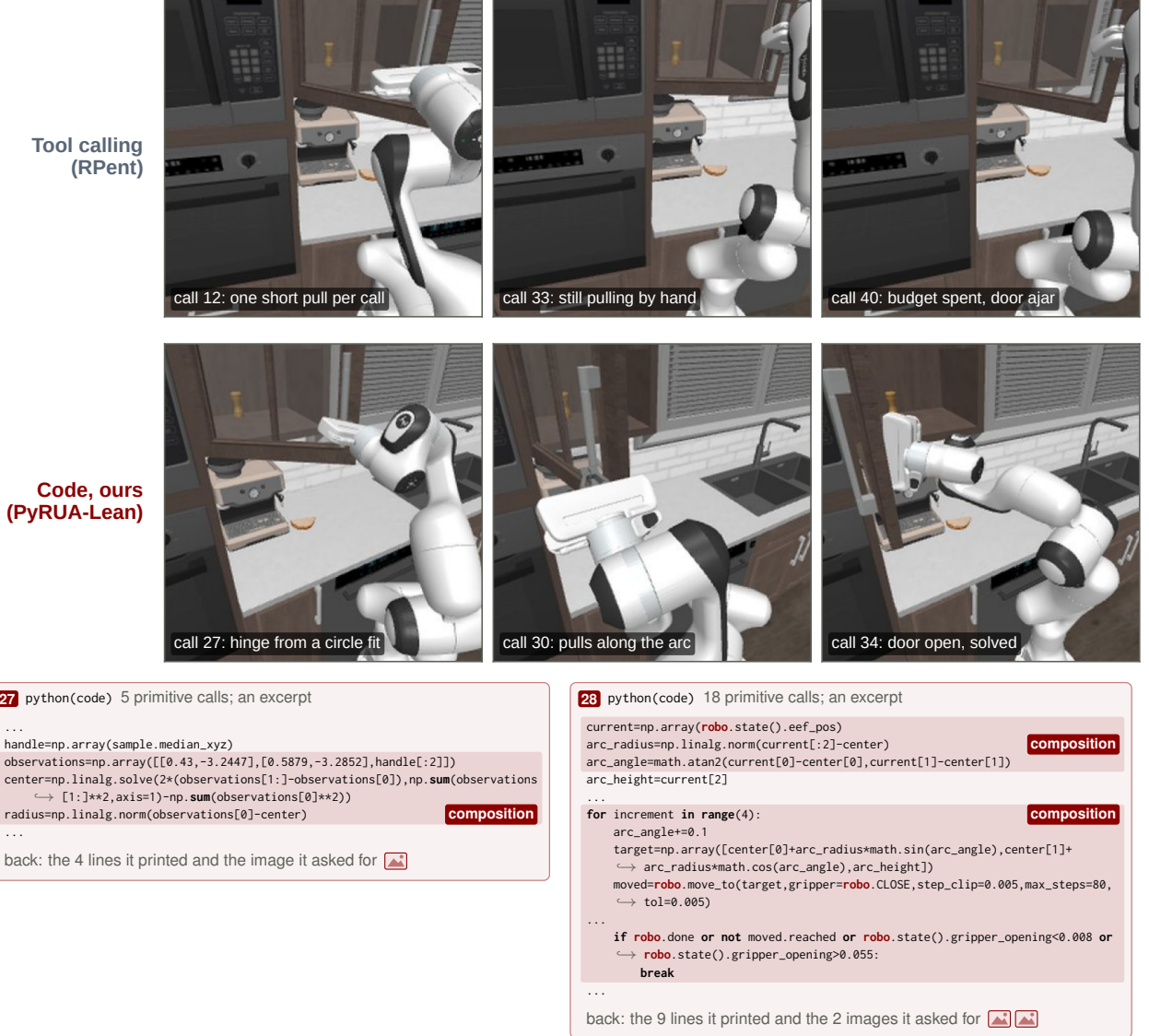


Figure 9 An instance only code solved: RoboCasa365 atomic, “Open the cabinet door.” (seed 4). Both agents located the door’s hinge. Tool calling pulled the handle a few centimeters per LLM call and was still pulling when its 40 calls ran out; the code agent took the hinge as the centre of the circle through three positions of the handle, two of which it copied from earlier outputs, and pulled along that arc in one cell, stopping if a move fell short or the grip slipped. The door opened, and a VLA call in call 34 completed the task. *Top*: the recorded camera views; *bottom*: the code agent’s cells, excerpts exactly as the model wrote them.